

Agile product line planning: A collaborative approach and a case study

Muhammad A. Noor^a, Rick Rabiser^b, Paul Grünbacher^{b,*}

^a *Institute for Systems Engineering and Automation, Johannes Kepler University, 4040 Linz, Austria*

^b *Christian Doppler Laboratory for Automated Software Engineering, Johannes Kepler University, 4040 Linz, Austria*

Received 27 March 2007; received in revised form 19 July 2007; accepted 10 October 2007

Available online 17 November 2007

Abstract

Agile methods and product line engineering (PLE) have both proven successful in increasing customer satisfaction and decreasing time to market under certain conditions. Key characteristics of agile methods are lean and highly iterative development with a strong emphasis on stakeholder involvement. PLE leverages reuse through systematic approaches such as variability modeling or product derivation. Integrating agile approaches with product line engineering is an interesting proposition which – not surprisingly – entails several challenges: Product lines (PL) rely on complex plans and models to ensure their long-term evolution while agile methods emphasize simplicity and short-term value-creation for customers. When incorporating agility in product line engineering, it is thus essential to define carefully how agile principles can support particular PLE processes. For instance, the processes of defining and setting up a product line (domain engineering) and deriving products (application engineering) differ significantly in practices and focus with implications on the suitability of agile principles. This paper presents practical experiences of adopting agile principles in product line planning (a domain engineering activity). ThinkLets, i.e., collaborative practices from the area of collaboration engineering, are the building blocks of the presented approach as they codify agile principles such as stakeholder involvement, rapid feedback, or value-based prioritization. We discuss how our approach balances agility and the intrinsic needs of product line planning. A case study carried out with an industrial partner indicates that the approach is practicable, usable, and useful.

© 2007 Elsevier Inc. All rights reserved.

Keywords: Product line engineering; Agile methods; Collaboration engineering; Product line planning

1. Introduction

An increasing number of organizations are shifting to a software product line (PL) approach to reduce time-to-market and development costs while at the same time increasing reliability and easing maintenance (Clements and Northrop, 2001). These benefits are realized through systematic reuse of common functionality among the products of a product line. Common functionality is provided via reusable core assets such as software components. Product line engineering comprises two key processes: domain engineering, i.e., planning, designing and implementing an infrastructure of reusable core assets, and

application engineering, i.e., deriving a specific product from this infrastructure by utilizing the reusable assets. Agile approaches offer similar benefits but through different means such as incremental and code-centric development, customer involvement, stakeholder collaboration, delayed design decisions, or intensive customer feedback. Agile methods are particularly useful in cases where the requirements or the environment change frequently.

It is interesting to explore whether these – at first glance incompatible – approaches can be reconciled. One would expect that agile principles are less relevant in PLE. For instance, as many products depend on particular core assets it is difficult to modify these assets quickly to avoid numerous side effects. However, it seems intuitively reasonable that making PLE more agile can open vast opportunities for organizations in form of new markets, customers, and products. Agile product line engineering, i.e., making product

* Corresponding author.

E-mail addresses: man@sea.uni-linz.ac.at (M.A. Noor), rabiser@ase.jku.at (R. Rabiser), paul.gruenbacher@jku.at (P. Grünbacher).

lines more responsive to ever changing customer needs or market developments, is a research topic deserving greater attention from the research community than in the past.

Carbon et al. (2006) explore the possibility of making application engineering activities more agile and conclude that such an approach is possible and potentially fruitful. Other authors have shown that rapid iterative development is possible in PLE by simulating the derived product allowing frequent feedback from users (Rabiser and Dhungana, 2007). Making domain engineering more agile is challenging due to the model-driven nature of this process (Tian and Cooper, 2006). In domain engineering models are used to describe product maps, features, or the variability of architectural elements. While there is no doubt concerning the value and usefulness of these models, building and maintaining them effectively remains challenging, regardless of the modeling approach used. Adopting agile values and principles to support the definition of such models is therefore very promising. In this paper we thus present a *product line planning approach* that complies with key agile values and principles. Product line planning (also termed as PL scoping) is the starting point of the product line development life cycle (Schmid, 2002a). The PL planning process aims at aligning business objectives with the technical solution informed by business case and market analyses. The desired output is a product map defining the scope of the individual products (Clements and Northrop, 2001) and giving a concise overview of a product line (Schmid, 2002b). It is usually defined at feature level and serves as a tool for communication among different stakeholders. The product map is continuously updated and revised to reflect the increasing understanding about the business domain throughout product line development.

The Agile Manifesto¹ emphasizes the importance of “individuals and interactions”. In PL planning mutually agreed solutions can only be achieved through close collaboration and knowledge sharing, two important agile principles. A product line engineer has to deal with often diverse viewpoints and concerns of the different stakeholders and a high number of issues that have to be addressed. He will benefit from an approach that provides guidance and helps to avoid unfocussed discussions. According to agile methods such guidance should not be constraining the participants by impeding their interaction. This requirement is well supported by collaboration engineering (CE), an approach to designing work practices for high-value collaborative tasks (Briggs et al., 2003). CE defines proven patterns of group collaboration (called thinkLets) that are used to compose collaborative processes (Briggs et al., 2003). EasyWinWin (Briggs and Gruenbacher, 2002) is an example of a thinkLets-based approach.

Our product line planning approach proposes the use of thinkLets to collaboratively develop a product map as proposed in Schmid (2002a,b). We carried out two initial fea-

sibility studies as briefly reported in Noor et al. (2006) and Noor et al. (2007a) to validate our approach. While the first study was targeted at assessing the suitability of a Group Support System to support a collaborative product line planning process (Noor et al., 2006), the second study aimed at testing the process itself (Noor et al., 2007a). However, these two studies were limited as they were conducted in the lab using a fictitious scenario. In this paper we provide a real-world case study conducted with a company and report on practical experiences. We also discuss the incorporation of agility in greater detail.

The remainder of this paper is structured as follows. Section 2 presents existing work in PLE and CE and work on incorporating agile principles into PLE. In Section 3 we present our collaborative product line planning process. Section 4 reports practical experiences and results from an industrial case study we conducted to validate our approach. Section 5 discusses the use of agile principles and values in the presented approach. We end the paper with a conclusion and an outlook on future work.

2. Existing work

We discuss related work on product line engineering with a focus on product line planning, existing research on incorporating agile principles into product line engineering, and work on collaboration engineering.

Fig. 1 depicts the relationship of our PL planning approach to existing methods and practices from different disciplines. The approach is based on well-established PL scoping approaches such as PuLSE Eco (Bayer et al., 1999b) and the guidelines of the SEI product line framework.² It incorporates relevant agile principles and uses thinkLets to facilitate collaboration in PL planning.

2.1. Product line engineering and product line planning

A huge body of work exists on PLE *foundations and methods*. For instance, Clements and Northrop (2001) discuss general concepts of product line engineering and identify key practices and process areas. Pohl et al. (2005) describe a framework for product line engineering processes and explain how to define a product line using orthogonal variability models. Bosch (2000) provides key insights on software architecture and product line architecture development. Northrop (2004) presents a product line adoption roadmap that helps visualizing the adoption process, identifying essential tasks and highlighting important focus areas. PuLSE (Product Line Software Engineering) (Bayer et al., 1999b) and Kobra (Atkinson et al., 2002) are examples of comprehensive PLE methodologies.

Product lines are typically not developed from scratch in organizations. Rather, companies with successful products in the market adopt a product line approach as soon as main-

¹ <http://www.agilemanifesto.org>.

² <http://www.sei.cmu.edu/productlines/framework.html>.

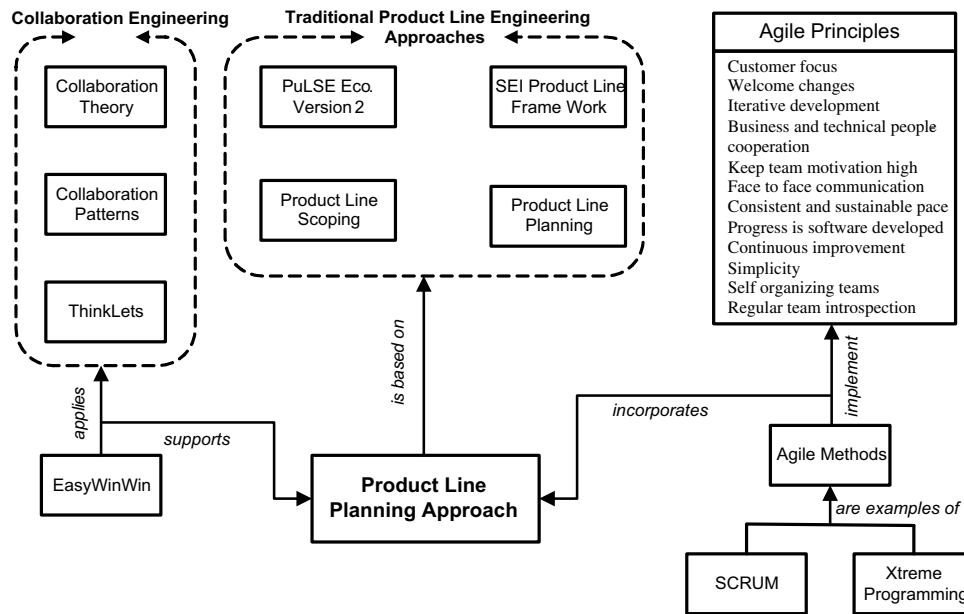


Fig. 1. Sources of our collaborative product line planning process.

taining a family of largely similar products becomes too costly and time-consuming (Bayer et al., 1999a, 2004; Clements and Northrop, 2001; Ebert and Smouts, 2003; Peterson, 2003). Product line adoption under such circumstances is referred to as *reengineering-driven product line development* (Schmid and Verlage, 2002). Existing systems are the result of large investments. They embody substantial domain knowledge and expertise and can not be easily discarded. The reuse of existing assets is therefore critical in PL development as developing existing systems anew for a PL is typically prohibitively expensive and risky (Aversano and Tortorella, 2004). Achieving systematic reuse in reengineering-driven product line development requires careful planning (Bosch, 2000; Clements and Northrop, 2001; Pohl et al., 2005). Existing assets such as software components, test cases, or documentation have to be analyzed thoroughly and adapted before they can be reused (Stoermer and O'Brien, 2001). Typically, many different individuals have been creating and maintaining these assets over many years. When scoping and defining a product line based on existing products it is therefore imperative to involve the people who are familiar with these assets (Noor et al., 2007a).

Several authors focus particularly on *product line planning*. Schmid and Verlage (2002), for instance, explore the economic impact of product line adoption and evolution and suggest the four adoption strategies big bang, project integration, reengineering-driven and leveraged. Schmid defines product line scoping as a three-stage process comprising product mapping, domain potential analysis, and reuse infrastructure scoping (Schmid, 2002a,b). For organizations having successful products in a particular domain, product mapping does not remain predominantly a business and marketing concern. It is also important to extract features from existing products along with higher-level architectural information. Instead of a domain potential

analysis, a reuse potential assessment of legacy assets needs to be conducted to reuse existing experience in a particular domain (Noor et al., 2007b). Legacy assets to be reengineered need to be identified and transformed into reusable components in a core asset base (referred to as reuse infrastructure scoping).

2.2. Incorporating agile principles into PLE

Tian and Cooper (2006) compare agile methods with product line engineering and show that it is difficult to use existing agile methods on an *as-is* basis due to the plan-driven nature of software product line approaches. However, they conclude that adding agility to product line engineering can be highly beneficial. Carbon et al. (2006) propose a broader definition of agility and argue that agile principles are easier to incorporate in product line engineering than directly applying well-defined existing agile methods. The authors show how agile principles can be incorporated into application engineering using Fraunhofer's PuLSE-I approach. Navarrete et al. (2005) show how agile principles can be adopted in COTS-based development, specifically in COTS selection. Boehm (2002) explores plan-driven and agile methods and their commonalities in general. He concludes that each method has its benefits and drawbacks and suggests hybrid approaches that combine the best of both worlds. This view is confirmed by Turner and Boehm (2003) who suggest to find a good balance between plan-driven and agile methods.

2.3. Collaboration engineering

Teams working together in pursuit of shared goals are an integral part of modern business environments. Many mission-critical tasks can not be performed by individu-

als because of their complexity and increasingly tight schedules. Achieving effective collaboration and exploiting the full benefits of collaboration is challenging (Briggs et al., 2001). Challenges can be seen at two levels: (i) when defining the collaborative process (at “design-time”), i.e., establishing objectives, defining tasks and their sequence, selecting participants and tools, etc.; and (ii) when enacting the collaborative process (at “run-time”), i.e., maintaining focus, guiding the discussion, or managing tools. Collaboration engineering is an approach to designing work practices for recurring high-value collaborative tasks by designing and deploying repeatable team processes at these two levels through *thinkLets* (Briggs et al., 2003). Briggs et al. (2003) define a *thinkLet* as “a codified facilitation technique that creates predictable patterns of collaboration”. A *thinkLet* is a named, scripted, and well-tested activity that produces a known pattern of collaboration among people working together on a common goal (de Vreede et al., 2006). *ThinkLets* were first proposed in (Briggs et al., 2001) and have become widely accepted entities for building collaborative processes. There are currently about 70 well-documented *thinkLets* (Kolfshoten et al., 2004). See the Appendix for an example. Complete details on *thinkLets* can be found in (Briggs and de Vreede, 2001).

Collaboration engineering identifies six general patterns of collaboration: generate, reduce, clarify, organize, evaluate, and build consensus (Briggs et al., 2006). Any *thinkLet* will generate some variation of one or more of these. In *generate* a group moves from having fewer to having more concepts with which to work (e.g., through brainstorming). In *reduce*, the group moves from having many to focusing on a few concepts deemed worthy of more attention (e.g., through facilitated discussions). In *clarify* people move from less to a more shared understanding of concepts and labels (e.g., through a moderated discussion about brainstorming results). *Organize* means that a group derives understanding of the relationships among concepts (e.g., structuring brainstorming results into different categories). *Evaluate* patterns increase understanding of the instrumentality of concepts (e.g., voting to understand the preferences of stakeholders). *Build Consensus* patterns move toward having more individuals willing to commit to a proposal.

ThinkLets have also been used to define collaborative approaches in software engineering. For instance, EasyWinWin (Briggs and Gruenbacher, 2002) is a groupware-supported requirements negotiation method based on Boehm’s Theory-W (Boehm and Ross, 1989). The EasyWinWin moderator guidebook is an example of how to design and conduct collaborative processes (Grünbacher, 2000). Grünbacher and Hofer (2002) show that *thinkLets* used in EasyWinWin also support agile methods by fostering the attainment of a shared vision and the involvement of success-critical stakeholders.

If desired, *thinkLets* can be supported by a wide array of collaborative tools. For instance, in our industrial case study (cf. Section 4) we used a Group Support System (GSS³) to illustrate and validate our PL planning approach. Web-based off-the-shelf collaboration tools such as Google Docs⁴ can similarly be used as our case study shows. It is however important to stress that CE and *thinkLets* do not rely on software tools. *ThinkLets* can also be enacted using conventional facilitation techniques such as flipcharts, cards, etc. Kolfshoten et al. (2006) discuss this technology-independent nature of *thinkLets* and argue that instead of prescribing a concrete tool and its configuration *thinkLets* should just define the capabilities required for later execution. While *thinkLets* might appear process-centric and tool-centric at first sight they should rather be seen as facilitation techniques described in a very compact form that allow structuring high-value group tasks with a focus on triggering the interaction of individuals and teams. *ThinkLets* can be combined flexibly to achieve the desired results in collaboration. For instance, different brainstorming and prioritization techniques can be concisely described using *thinkLets* to allow the rapid composition of collaborative activities later on. It is important to stress that collaborative practices such as the planning game in eXtreme Programming are an integral part of agile methods.

3. A collaborative PL planning approach

Stakeholder collaboration is a necessity rather than a matter of choice in product line planning as no individual stakeholder has all information necessary to assess the business value of product features and the technical feasibility to reuse certain assets in a product line (Noor et al., 2006). Achieving a balance relies on involving non-technicians (e.g., senior management, marketing, sales staff) as well as technical staff (e.g., architects, developers, engineers, maintenance staff, domain experts) (Noor et al., 2007a). The collaborative product line planning approach supports an organization intending to migrate towards a product line approach. The approach incorporates agile principles and guidelines from collaboration engineering (Briggs et al., 2003; Grünbacher, 2000; Kolfshoten et al., 2004, 2005). Agile product line planning as we perceive it in the context of this paper refers to the speed through which an initial product map can be developed, the flexibility and non-prescriptive nature of the process, the collaboration of business and technical people with face-to-face interaction as an important means of communication, the ability to embrace changes during the process, and the focus on technical excellence.

³ <http://www.groupsystems.com>.

⁴ <http://docs.google.com>.

3.1. Stakeholders

Agile methods emphasize the involvement and interaction of business and technical people in software engineering. This requires the intensive involvement of representatives from different groups who developed or are responsible for the existing products (Dhungana et al., 2006a). Following Boehm's spider web (Boehm et al., 2000) we describe the success-critical stakeholders for product line planning and summarize their typical concerns.

The decision to adopt a product line approach often comes from *senior management* which outlines business objectives and allocates time and resources for the project. Senior management is typically interested in cost-effective, timely, and robust solutions which deliver the desired results with minimal risk exposure. After the project start senior management has to ensure that the primacy of business objectives is not compromised due to technical concerns. In most cases it is not feasible or possible to directly involve single *customers* in internal product reengineering processes (i.e., extraction of core assets from existing systems) as product lines typically target a wide variety of customers. It is difficult but nevertheless necessary to involve a representative sample of customers or customer groups in product line planning representing the targeted market segment. Often, customer concerns and expectations additionally have to be ascertained and incorporated through other means. For example, *marketing and sales people* share their market knowledge and new requests of different customers. They frequently interact with actual users and customers and are essential to represent the business concerns in PL adoption. Their understanding of sales records and market trends allows them forecasting future sales and assessing expected benefits. In product line engineering we have to understand and reconcile the needs of different customers and customer groups. Instead of an "on-site customer", a more comprehensive, collaborative approach to PL planning is needed to attain mutually satisfactory agreements. If legacy products are to be reused in product line development, the involvement of *software architects* becomes necessary along with developers and maintainers of those products (Dhungana et al., 2006b). There are many inherent complexities in reusing legacy products in product line development, for example when independent individual legacy products are to be merged in a single platform (PL infrastructure). There may be significant differences in terms of product architecture and implementation technology along with numerous combinations of differences in non-functional attributes. *Developers* can assess different alternatives of legacy assets from a reusability perspective. Their knowledge facilitates estimating the costs of changing different aspects of legacy products. *Maintenance staff* is highly valuable at the early stages of product line planning due to its intimate knowledge of the strengths and weaknesses of existing products. Feedback from maintenance staff enables the evaluation of different PL evolution options to ease transition to customers and to ensure product compatibility.

3.2. Tasks

The objective of our approach is to define a product map which is prioritized at feature, domain, and product level. Features are externally visible characteristics of a system (e.g., "save project"). A product map can be visualized as a feature-product matrix where each feature is part of exactly one domain representing a coherent cluster of functionality (Schmid, 2002a).

Our process consists of five tasks (cf. Fig. 2):

- (1) *Identify and agree on relevant domains.* Stakeholders jointly review existing domains and expand/revise the domains for further planning. The result of this step is a mutually agreed list of domains for the planned product line.
- (2) *Define features for each domain.* Stakeholders brainstorm features for the identified domains, i.e., new features or features from existing products. The collected features are assigned to the domains.
- (3) *Discuss, analyze, and agree on products.* Stakeholders precisely define products by considering the results from the domain and feature analyses. This task may be based on an initial product list developed prior to the process by business and marketing people (Schmid, 2002a).
- (4) *Define products in terms of features.* Stakeholders relate features to products in the product line to create an initial product map.
- (5) *Prioritize product map.* Stakeholders evaluate the business value of features and the feasibility to realize them. The result of this step is a product map with all features prioritized according to their perceived business value and feasibility.

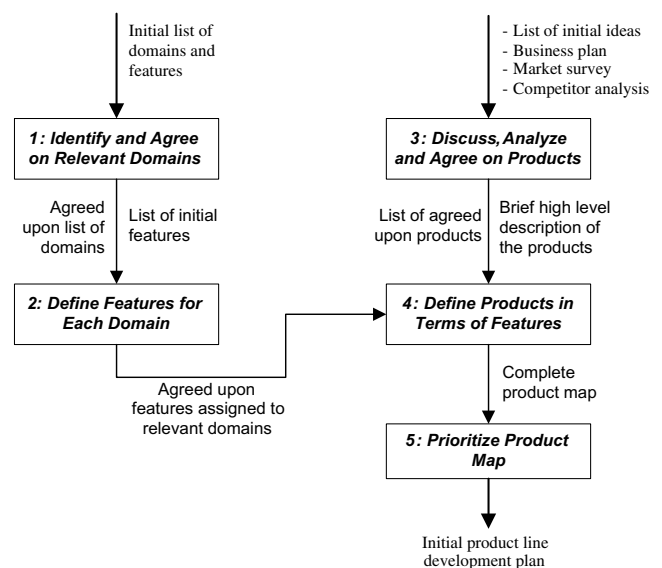


Fig. 2. Collaborative PL planning approach.

The five tasks are performed in an iterative manner to allow frequent feedback. For instance, it is common in task 3 that new domains are identified during the discussion of the products which will require the collaborative collection of features for these new domains (task 2). It may also happen in task 5 that as a result of the prioritization a domain is considered obsolete and has to be deleted from the final product map. We will discuss further examples of process variations in Section 4 when describing the case study with industry partner PROSOL. Fig. 3 shows the suggested thinkLets for each task together with the collaboration pattern of the thinkLets. The process uses nine unique thinkLets for the five tasks. Again, there is leeway for the facilitator when organizing the collaboration. ThinkLets can be dropped or used in a different sequence when conducting the approach.

We will now explain the tasks and how they are supported by thinkLets. For brevity's sake we limit the

detailed description of the instantiated thinkLets to tasks 2 and 4:

Task 1 (*Identify and agree on relevant domains*). The input to this task is an already prepared list of domains. The goal of task 1 is to review and agree on a list of domains. The focus is on achieving a common understanding of the existing domains rather than creating new domains. Participants walk through the list, mutual discussion ensures common understanding of the domains. The thinkLets *DealersChoice* followed by *FastFocus* (cf. Fig. 3) are used in this task. It is important to limit the time spent on this task as the list of domains can be updated at any time later in the process.

Task 2 (*Define features for each domain*). The inputs to this task are the domains from the previous step and existing lists of features. The team organizes the features into the domains identified using the thinkLet *PopcornSort*. Participants may also suggest new features for these

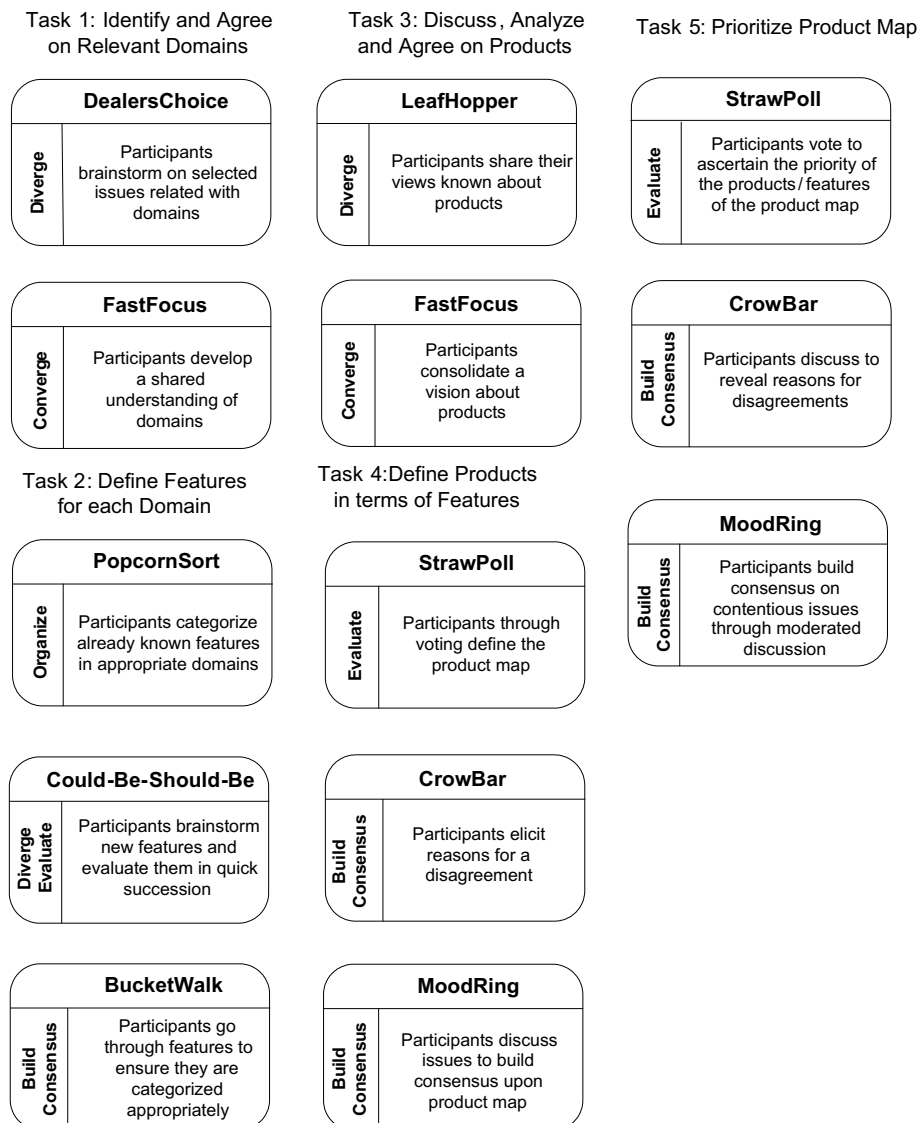


Fig. 3. Recommended thinkLets in the collaborative PL planning process.

domains. However, as using a free brainstorming technique would result in too many features requiring significant time to discuss them, it is preferable to use the thinkLet *Could-Be-Should-Be* instead. Participants brainstorm only a few features for each domain and are asked by the facilitator interactively to evaluate whether these features should be part of the domain. Brainstorming and evaluation can be repeated as often as needed. Finally, using the thinkLet *BucketWalk* the team removes redundancy and consolidates features for each domain. Table 1 shows details of the instantiation of thinkLets for task 2.

Task 3 (*Discuss, analyze, and agree on products*). Marketing and sales people, in consultation with senior management, prepare a list of desirable products as input for this task on the basis of market studies and analyses. The list of products must be shared with the team to achieve mutual understanding. The main aim of this task is to ensure that defined products are desirable for the customers. Customer involvement is thus especially important for this task. The thinkLet *LeafHopper* allows collecting thoughts and ideas of team members about possible products. Ideas expressed about the products need to be organized and promising ideas are incorporated into the product description. This description can be compared to the concept of a product metaphor often used in agile methods. For instance, in the case study described in Section 4 we used a question set (cf. Table 3) to facilitate product concept development. For the selection of the most promising ideas the team relies on the thinkLet *FastFocus*.

On the basis of the most promising ideas the team then broadly defines the products so that features can be assigned to products unambiguously in the next task.

Task 4 (*Define products in terms of features*). This task offers the opportunity to revisit any issue covered in the previous steps. So far participants have assigned features to domains and agreed upon products. The *StrawPoll* thinkLet is used to assign features to products. Participants get a ballot in tabular form with the features listed in rows and products listed in columns. According to the chosen voting method (e.g., a 1–10 scale) they decide whether a particular feature should be part of a product. Voting results are tabulated and the statistically significant difference of opinion regarding features being part of certain products is displayed (e.g., by marking cells red). To ascertain the reasons behind difference of opinion thinkLet *CrowBar* is used. When all bones of contention are known, *MoodRing* helps the team to reach consensus on an initial product map. Table 2 shows details of the instantiation of thinkLets for task 4.

Task 5 (*Prioritize product map*). The input to this task is the initial product map developed as output of task 4. Products and features are prioritized from the perspectives of technical feasibility and customer desirability. Task 5 is divided into two subtasks: First the products are prioritized by performing the thinkLets *StrawPoll*, *CrowBar* and *MoodRing* (see Table 2); then the thinkLets are used again to prioritize features for each product. The output of the task is a mutually agreed and prioritized product map.

Table 1
Instantiated thinkLets for Task 2: define features for each domain

<i>PopcornSort</i>	
Input	List of agreed upon domains from previous step and already prepared feature list
Description	Provided features are discussed in case of ambiguity and are categorized in appropriate domains
Output	Features categorized in domains
Parameter	Find most appropriate domains for these features
Rules	A feature can be assigned to one domain only. Comments can be added to any item (domain/feature)
Actions	Drag items from feature list to domain buckets
Capability	Features and domains written on cards and on a board
<i>Could-be-should-be</i>	
Input	List of domains and features produced as result of PopcornSort
Description	Brainstorming and evaluation are performed in quick succession repeatedly in order to find new features and categorize features in appropriate domains and sub-domains. First, items which could be part of the PL are added and then they are evaluated and consolidated. Items on which there is consensus that they should be part of PL are kept in the list. The remainder is removed
Output	Enhanced list of features for each domain
Parameter	Add relevant new features which could be part of the product line. Select those features which should be part of product line
Rules	It is not compulsory for each person to add features in each domain; comments can be added to any item
Actions	Add new features for a particular domain, add comments
Capability	Same as in previous thinkLet PopcornSort
<i>BucketWalk</i>	
Input	List of features, sub-domains, and domains prepared in previous steps
Description	Participants go through the categorized list reviewing all items through moderated discussion. Focus is on desirability of the item and correct categorization
Output	Structured and agreed list of features, sub-domains, and domains
Parameter	Judge the feature on desirability and appropriateness and check if it was categorized correctly
Rules	Review item one by one in a moderated discussion
Actions	Edit and move items in hierarchy
Capability	Ability to move items in hierarchy, and to rephrase them

Table 2
Instantiated thinkLets for Task 4: define products in terms of features

<i>StrawPoll</i>	
Input	List of features, sub-domains and domains and list of products prepared in previous steps
Description	Participants assign features to products through voting. Diverse voting methods can be used. A 1–4 scale (strongly agree, agree, disagree and strongly disagree) is a good option
Output	Initial product map
Parameter	Technical feasibility and business value are the criteria to judge the suitability of assigning a certain feature to a product
Rules	Use anonymous voting
Actions	Fill in the ballot and cast the vote
Capability	Ballot, pencil, box to support anonymous voting
<i>CrowBar</i>	
Input	Tabulated results of voting done in previous step, showing agreements and disagreements of the team
Description	Through moderated discussion participants reveal reasons and assumptions behind disagreements without revealing how they have voted. The focus here is to pry out the causes of disagreement
Output	A mutually agreed product map
Parameter	None
Rules	Moderated discussion
Actions	To the point discussion
Capability	No technical capability is required here. People are required to talk in turns regarding issues of disagreements
<i>MoodRing</i>	
Input	Results of voting, highlighting disagreements and related causes, issues, and reasons behind respective disagreements, uncovered in CrowBar
Description	Participants take one issue (disagreement) at a time and start discussing the way to get around the disagreements. Each participant has the ability to change his vote if during the discussion things come up which change her mind. Discussion on disagreement will continue till consensus is achieved. This thinkLet is focused on building consensus
Output	A structured and agreed list of features, sub-domains, and domains
Parameter	None
Rules	Anonymity of vote change
Actions	To the point discussion
Capability	Ability to vote and revote anonymously and results of voting (or re-voting) should be computed quickly. This thinkLet can be performed without electronic tool. When the team thinks a revote is due (as sufficient differences have been clarified through discussion) a revote is conducted

4. The PROSOL case study

We present a case study conducted with our industrial partner PROSOL⁵ to present practical experiences and to validate the approach with quantitative and qualitative data. PROSOL is a small Austrian software firm specializing in the domain of supply chain management. Their main online services are E-Tendering, E-Procurement, and E-Shipment. PROSOL's products provide capabilities for issuing online tenders, negotiating tenders, and delivering goods through various transportation means. PROSOL offers a product for e-procurement in the logistics domain with customers in several European countries. The company intends to expand the range of products and services in the near future. In order to make best use of their experience the company will diversify their product portfolio in the supply chain management domain. The company intends to shift to a PL approach incrementally to learn essential technical skills necessary to build up and manage a PL throughout the shifting process. Furthermore, such an incremental PL adoption strategy will enable them to evaluate market response with minimal exposure to risk.

The goals of the case study are to present data regarding the effectiveness, efficiency, and usability of the process and

to report on practical experiences in conducting it. In particular we are interested (i) whether the productivity of the process is higher than the productivity of comparable team elicitation techniques used in software engineering; and (ii) whether the product map produced during the workshops is of sufficient quality. In Section 5 we then will also discuss in detail how the process supports agile principles based on an analysis of the recorded observations.

The product line planning process was conducted in two face-to-face workshops. The GroupSystems GSS³ was used to support the face-to-face group interactions. Four people from PROSOL participated in the workshops that covered all the roles discussed in Section 3.1. Two of the authors were in charge of running the process and served as facilitator and scribe. One author observed the process and was responsible for measurement. The process of refining the product map continued after the workshops at PROSOL. Stakeholders used the online spreadsheet capability Google Docs⁴ to evolve the defined product map. We observed this process and the data on the evolution of the artifacts produced (e.g., the feature map) over a couple of months, to derive conclusions on its effectiveness.

4.1. Conducting the process

The first workshop started with a brief introduction of the participants and their roles in the organization. Next

⁵ http://www.procurement-solutions.com/prosol/index_en.html.

the facilitator presented the collaborative product line planning process, its constituent tasks, and the purpose of each of these tasks. The GSS tools are easy to use and a dedicated training course was not necessary; however a brief introduction of the collaborative tools ensured a smooth start. About 25 min were spent for the initial demonstration.

The *first task* was to review and discuss already known domains. The input for this task was a list of domains prepared by PROSOL before the workshop that was made available to the group in the GroupOutliner tool depicted in Fig. 4. The purpose was to achieve mutual understanding among the team regarding domains.

In a moderated session participants expressed their understanding of each domain. The process initially started with six domains, however, as a result of discussion and better understanding a new domain (Interface) was added to the list. Later that list was changed again; in tasks 3 and 4, the B2C domain was removed because participants regarded it as a sub-domain of E-Tender. Furthermore, the

two domains Accounting/Billing and Communications were added in order to classify the features appropriately.

It was found unnecessary to explicitly brainstorm for additional domains as the participants were highly familiar with the PL scope and did not expect to discover new domains. The facilitator deviated from the plan and accomplished the task successfully using only the thinkLet *FastFocus* which demonstrates the flexibility of the process. At any stage during the workshop teams can suggest changes to the process. The facilitator can flexible combine some or all suggested thinkLets to address the suggestions. Accomplishing a task with fewer thinkLets is possible depending on the team's prior knowledge and the proceeding of the workshop.

The *second task* was to gather existing features for each domain. The team performed five brainstorming sessions using the electronic brainstorming tool shown in Fig. 5. In free brainstorming, sheets are swapped as soon as an idea is added to the sheet. Looking at other participants' submissions stimulates the generation of new

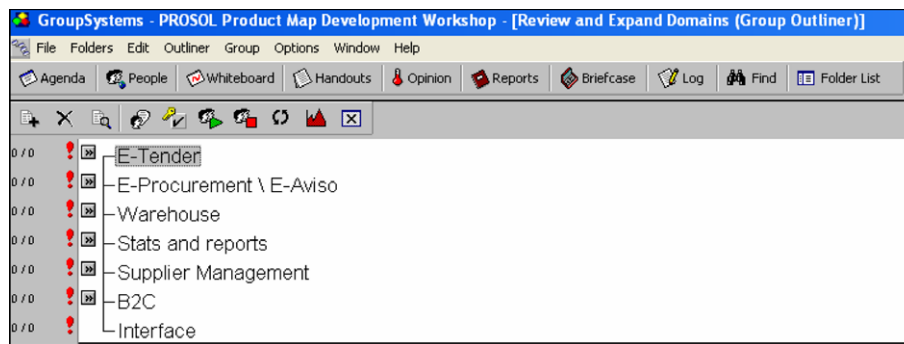


Fig. 4. Review and expand domains in Group Outliner (top domains shown only).

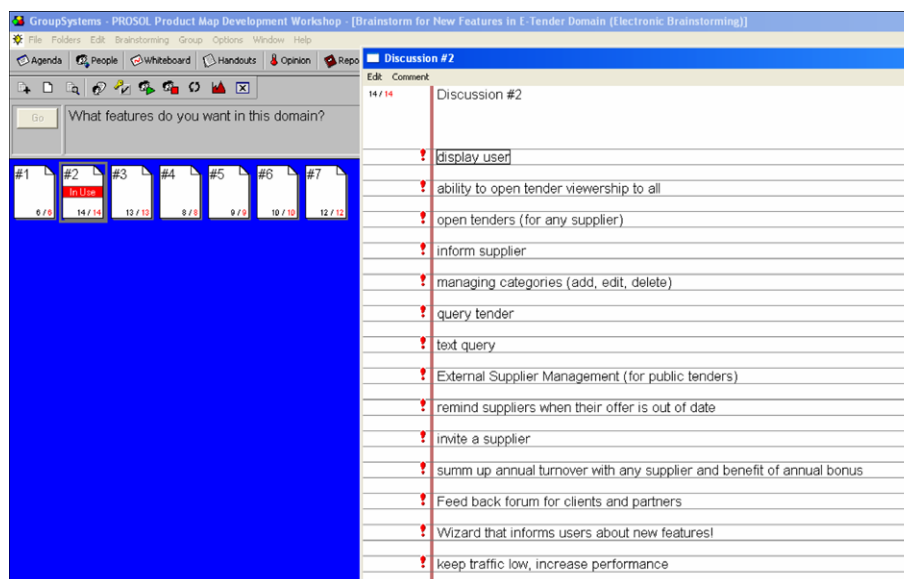


Fig. 5. Brainstorming for features in the E-Tender domain.

ideas. After each brainstorming session, elicited features were converged upon (thinkLet *Could-Be-Should-Be*). Through moderated discussion, features not belonging to the domain for which they were proposed were moved appropriately.

After brainstorming the candidate features for five domains the team realized that a sufficient amount of features for the new domain Interface had already been gathered and decided to conduct no further brainstorming session. The team did not perform *PopcornSort* to categorize the pre-prepared list of feature to domains as they directly went to brainstorming for new features. These run-time adaptations of the process further illustrate its non-prescriptive and non-rigid nature. The team then converged on features based on the complete list of candidate features generated through brainstorming. Fig. 6 shows the GSS tool Categorizer we chose to support this task. The purpose of the moderated discussion was to eliminate redundancies and ambiguities from the candidate features and to unambiguously specify the features in the different domains.

Moderated discussion is central to the task in order to guide the team and to maintain focus. Each participant was assigned one brainstorming sheet. As shown in Fig. 5

seven sheets were used in each brainstorming session. Participants were free to choose – within their sheet in use – to read a feature of their choice. Each participant picked an arbitrary feature from the list and explained it to the team in a round robin manner. He rephrased the feature if requested by other participants to eliminate ambiguities and to ensure common understanding. After one round all participants were assigned a new sheet. Our experience in this case study confirms the benefits reported of Easy-WinWin in (Briggs and Gruenbacher, 2002). The team converged on high-value features quickly and shared understanding of the features was established which demonstrates the synergies of a collaborative process. After brainstorming and converging upon features, the team used thinkLet *BucketWalk* to iterate through the list again to further refine it. As a result two more domains were identified (Accounting/Billing and Communications). The team also moved features to appropriate domains. In this phase changes are welcome at any time, however, in a controlled manner: If participants wanted to modify or move a feature they asked the facilitator who then carried out the change on their behalf if all team members agreed. This allowed maintaining a shared list of already agreed elements and performing changes when necessary.

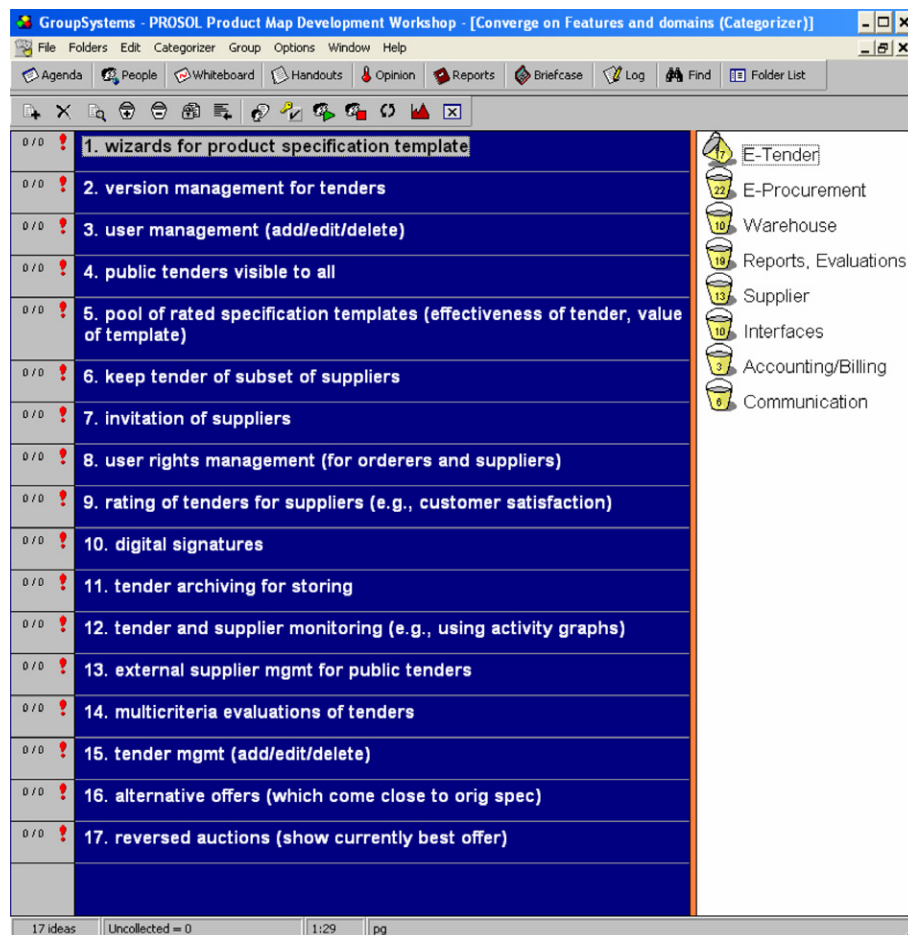


Fig. 6. Converging on features in the Categorizer tool.

The next task was conducted using a slight variation of the original plan which suggests conducting feature prioritization together with domain and product prioritization during task 5. During the workshop participants suggested to prioritize features earlier to maintain continuity. The input to the task was a consolidated feature list from the previous round. The aim was to first prioritize the features on the basis of business value and technical feasibility and then go through the feature list again to further consolidate it. The task was performed using thinkLet *StrawPoll*. Participants rated each feature on a 1–10 scale. Prioritization helped achieving a better understanding of the features because the team was required to relate with them by assessing their value. The transition from creativity (brainstorming) to analysis (consolidation and categorization) to evaluation (prioritization) was smooth and confirmed the feasibility of the process from practical perspectives.

In the *third task* the team discussed and agreed on products. This task is rather unrelated to the tasks we have performed so far as it does not take the output of previous tasks as an input (cf. Fig. 2). It is expected in usual settings that teams start this task with a list of existing products; which will require elicitation of details about the products through thinkLet *LeafHopper* and convergence about products through thinkLet *FastFocus*. However, that was not the case with PROSOL where the participants had only vague ideas about future extensions to their existing products. Variation in the process was required. We conducted a brief brainstorming session to elicit possible products. Initially 11 products were proposed by the team. The facilitator let people rank-order these proposed products. Subsequently, he used the questions shown in Table 3 to establish a common understanding regarding products and their possible scope.

The team initially answered these questions for the four top-ranked products. In order to save time each member was working on one product of his choice. Later the participants read each others answers and in a moderated discussion each member revealed what he had written and, when necessary, gave a rationale. We used the free brainstorming to collect initial products and then a slight variation of thinkLet *LeafHopper* to collect the answers of product-related questions. Participants were assigned a product form they were asked to complete. In its original form

LeafHopper requires that participants are free to choose which questions they want to answer. *StrawPoll* was then conducted to rank-order the products. The participants had identified 11 products and converged on 100 features. In order to create a product map consisting of 11 products each participant would have had to vote on 1100 items. To save time the facilitator therefore decided to only vote on the top four ranked products during the workshop.

The *fourth task* was to actually define the product map. The input to the task was the list of consolidated and prioritized features, domains, and products from tasks 2 and 3. We used the tool *AlternativeAnalysis* to categorize the features to appropriate products. The team agreed that the top four products ranked in the previous task are enough for immediate future development, so we used only these products to define the product map. We used thinkLet *StrawPoll* with a 4-scale voting method (strongly agree, agree, disagree, strongly disagree). There were only a few disagreements about which features should be part of the different products. PROSOL members discussed them internally and agreed upon the product map. It was thus not deemed necessary to use the thinkLets *CrowBar* and *MoodRing*.

Part of *task 5* had already been conducted between tasks 2 and 3 (as explained above). Task 5 continued after the workshop using a Google Spreadsheet document⁴ to update and review the final product map. Since this is a time-consuming activity that can be conducted without a moderator by the company alone, it was more convenient to carry it out in a distributed manner.

4.2. Results

The PL planning process was carried out without any major disruptions. For preparation, the industrial practitioners prepared a list of already known features and domains which took them only a couple of hours. The facilitator prepared the agenda and demonstration of the GSS tool in about 3 h. A brief presentation of the objectives of the workshop along with methods was sufficient to kick-start the process. The facilitator needed less than 25 min to explain the process, briefly introduce the product line context, and demonstrate the tools used in the workshop. There was no major confusion or misunderstanding regarding the objectives of the workshop. We investigated two research questions regarding the productivity and efficiency of the process.

4.2.1. Is the productivity of the process higher than the productivity of similar team elicitation techniques used in software engineering?

Table 4 reports metrics for brainstorming and converging upon features. The planned duration of brainstorming and converging upon elicited features was 20 and 10 min, respectively. However, as Table 4 shows, the first round of brainstorming and converging took 56 min in total while later rounds took much less time. The duration for both

Table 3
Question set to elicit product details

#	Product-related questions
1	Briefly describe the product in a sentence or two
2	Does the proposed product fully or partly already exist, if yes to what extent?
3	Roughly how much effort would be required to implement the product?
4	Are there any similar products available in the market?
5	Who would be the potential customer?
6	Is this product viable business-wise?
7	When would you like to implement this product?

Table 4
Productivity of the tasks brainstorming and converging upon features

Domain	# Brainstormed features	Duration (min)	# Converged feature	Duration (min)
E-tender	72	26	17	30
E-procurement	41	20	22	15
Warehouse management	29	12	10	11
Reports	41	12	19	10
Supplier management	32	9	13	10
Interfaces	x	x	10	x
Accounting/ billing	x	x	3	x
Communication	x	x	6	x
Total	215	79	100	76

tasks averages out in five rounds to 30 min. A total of 215 features were collected in the five brainstorming sessions and 100 were converged upon in the moderated sessions. The convergence rate out of elicited features was 47%.

The features were categorized in eight domains aptly representing the business area of PROSOL. Feature and product prioritization have helped the team to remove less promising ideas early on and to focus on those areas that are most valuable for PROSOL's customers. The team also converged on four products (based on 11 product suggestions) and assigned the 100 features to the products to come up with the product map. All these tasks were accomplished in two half-day workshops. As a crude estimate of efficiency we decided to use the features generated per hour of stakeholder participation. Data indicates that 4.2 features were identified per hour of stakeholder participation. This is much higher compared to data reported from other requirements elicitation methods: For instance, Maiden et al. (2007) report that only 1.1 requirements were generated per hour of stakeholder participation in workshops for scenario-based requirements discovery. A major reason for the high productivity of our process in the case study is the familiarity of stakeholders with the domain as parts of the features were already being implemented at the time of the workshop. However, even if considering that only about 50% of the identified features were new, the productivity is twice as high compared to the data reported in the literature.

4.2.2. Is the product map produced during the workshops of sufficient quality?

Determining the quality of the product map developed during the workshops is not straightforward as traditional quality dimensions like correctness, consistency, and completeness are very hard to assess immediately after the workshop. However, PROSOL elaborated the product map in the months after the workshops using a Google Spreadsheet document that allowed distributed editing. We analyzed the evolution of the initial product map in the first 5 months after the workshop to determine the

quality of the workshops' results. The rationale is as follows: A high number of deleted or added features would indicate an insufficient quality of the initial product map as many changes are necessary to match the expectations of stakeholders. The evolution of individual features indicates the addition of details and refinements to clarify the meaning of the features.

Table 5 shows that from the 100 features identified in the product map only 11 were rephrased to clarify their meaning or to add details. Only two features were moved from one domain to another showing that the initial assignment to domains was valid to a large extent. Only three new features were identified after the workshops when elaborating the product map. Eight features were dropped from the product map as they were no longer considered relevant. Table 5 details these numbers for the different domains of the product map. The domains remained unchanged after the workshop. The small degree of evolution with a focus on rephrasing and refining features shows the high consistency of the product map and real stakeholder preferences.

The fact that intermediate results of each task were continuously evaluated and refined during the workshop certainly contributed to these positive results. For example, brainstormed features were first refined during convergence, then through implementation of thinkLet *Bucket-Walk* in feature prioritization, and lastly while assigning features to the products in product map development. However, it is important to note that since the time of workshops, PROSOL had already implemented at least 70% of the features identified as new during the workshops and the rest of the features are already in a pipeline. On the basis of the above-mentioned observations and data we are confident that the quality of artifacts produced is sufficient for the tasks at hand.

Clearly the focus of this paper was to report practical experiences of applying this method in real-world settings. We are aware about the limitations of the validation. First we do not compare results of our collaborative approach with traditional product line scoping approaches due to a lack of published empirical results. Also we only applied

Table 5
Evolution of the PROSOL feature map

Domain (# of features right after workshop)	Rephrased	Moved	Added	Deleted
E-tender (17)	4	1 to Reports	0	2
E-procurement(22)	4	1 to Reports	0	3
Warehouse management (10)	1	0	0	0
Reports (19)	2	2 moved here	1	2
Supplier management (13)	0	0	0	1
Interfaces (10)	0	0	0	0
Accounts/billing (3)	0	0	0	0
Communication (6)	0	0	2	0
Totals (100)	11	2	3	8

the presented approach in one case study. We are however confident due to our earlier experiences with thinkLets-based methods that the results would be similar in other cases.

5. Discussion: agility of the approach

We discuss the relevancy of the four agile values from the Agile Manifesto and show how they are supported by our approach. We then describe to what extent the approach addresses other relevant agile principles from the Agile Manifesto.

The agile value of “*individuals and interactions over processes and tools*” is highly relevant from the perspective of PL planning. We leverage experience and knowledge of technical and business stakeholders to achieve mutually agreeable plans through face-to-face collaboration. In order to facilitate effective and efficient collaboration we use thinkLets developed in the field of CE. While we also provide a high-level process and discuss possible tool support, we explicitly emphasize stakeholders and their interactions. The approach considers as more important what is happening in the group than what is happening in the tools.

The agile value of “*working software over comprehensive documentation*” is difficult to achieve fully in our context as various models and documentation are required to achieve a high level of reuse. However, in the context of reengineering-driven product line adoption, existing software existing provide the starting point for developing a product map (Noor et al., 2007b). Instead of merely using existing documentation, PL adoption requires extracting knowledge about existing products from the people who developed them.

The agile value of “*customer collaboration over contract negotiation*” is particularly interesting in PLE. Instead of a single customer it requires the involvement of representatives from multiple different customers and/or customer groups. For example, a group of key customers in the market segment of interest has to be involved as the product line aims to satisfy a wide range of customer needs. It is also essential to involve marketing and sales people in the product line planning process typically aided by market analyses, customer interviews, and market surveys. The presented approach assumes the participation of diverse stakeholders in PL planning (cf. Section 3.1).

The value of “*responding to change over following a plan*” is achieved by designing a flexible collaborative PL planning process. We suggest a high-level process that does not prescribe a strict sequence of steps or a concrete tool to be used. This is especially important and necessary to address issues that come up on-the-fly during stakeholder interactions. It is apparent from the accounts of the case study that our process provides a high degree of flexibility. The sequence of the steps was changed along with constituent thinkLets quite often throughout the two workshops. It also became evident that in the steps of categorization,

convergence, prioritization, and evaluation, features and domains were frequently changed and modified. We are therefore confident that our process supports well the agile principle “*...embrace change...*”. The principle “*welcome changing requirements...*” has greater significance in application engineering than in domain engineering. However, changes can be made to the product map while developing core assets in domain engineering as well. PL planning and core asset development can, and in fact often are, conducted in an iterative manner. For instance, in the workshops we conducted, participants frequently suggested new features or re-prioritized features any time they wanted to.

The continuous in-process quality checks support the agile principle “*continuous attention to excellence*”, as, e.g., redundancies were removed and refinements were made continuously throughout the process. The agile principle “*At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behavior accordingly (regular team introspection)*” is built into the proposed planning process. At each step, work done is reviewed and refined through categorization, converging on key elements, evaluation of ideas, and prioritization.

The agile principle “*...face-to-face conversation...*” is a cornerstone of our approach. The two workshops of the case study demonstrate that we rely on face-to-face conversations, e.g., when converging on the features in a moderated discussion. The principle “*build project around motivated individuals...*” is incorporated in our approach insofar as we recommend a careful team selection. Selecting people with a maximum of knowledge and experience does not automatically lead to motivation; you also need to have an eye on their level of motivation. The selection of stakeholders is essential as exemplified in Boehm’s CRACK (Collaborative, Representative, Authorized, Committed, Knowledgeable) criteria (Boehm and Jain, 2005).

The principle “*deliver working software frequently...*” is significant in application engineering and can also be applied in domain engineering where core assets are developed iteratively; but has little relevance to the product line planning activity per se. The same arguments apply to the agile principle: “*working software is the primary measure of progress*”.

The principle of “*simplicity*” is hard to achieve in PL planning in an absolute sense. We interpret this agile principle as “not doing unnecessary work”. The prioritization of domains and features is essential to keep the list of features small and to prune it whenever possible. Simplicity could be even increased by dividing complex tasks to avoid cognitive overload. For example when participants voted for the product map, there were 100 features and four products. That means they had to make 400 decisions in a short period. In order to enhance the quality of decisions participants suggested conducting several short workshops instead of one or two marathon workshops. This reduced the cognitive load and gave the participants time to digest the ideas.

6. Conclusion and outlook

Although many aspects of domain engineering seem to favor a rigorous and plan-driven working style, the experiences reported in this paper show that incorporating agile principles into product line planning is feasible. The results confirm the views of Carbon et al. (2006) that agile product line engineering is feasible. However, it may not resemble fully with existing agile methods as not all agile values and principles are directly relevant. We specifically dealt with the PL planning activity and explored how agile it can become, similar to existing work by Navarrete et al. (2005) in the area of COTS selection. Our PL planning process incorporates principles of agility that are most relevant in this context. The case study confirms that the process is effective, efficient, and usable. The presented process is collaborative and relies mostly on face-to-face conversation as a means of communication. It requires involvement of both business and technical people. We believe the presented process can serve as an example of maintaining balance between agility and more disciplined processes in software engineering.

We identified three success-critical issues regarding stakeholders: (i) it is important that participants are familiar with fundamental concepts of product line planning to allow a smooth start. (ii) Preparation is required on part of the process facilitators but also on part of the industry partner. (iii) The willingness to collaborate in a team is also essential. Despite direct customer involvement is not always feasible we have seen that in such cases representatives for target customer groups can be involved in PL planning.

Although we have applied our approach in a comparably small case study we believe that it will scale well to larger problems. It has been shown that chances of success do not diminish with an increasing number of participants (Briggs and Gruenbacher, 2002; Briggs et al., 2003). Also, similar collaborative processes have been used successfully in related disciplines such as requirements negotiation (Briggs and Gruenbacher, 2002; de Vreede et al., 2006). We will apply our process on a larger scale in the future to assess its scalability. We will also modify the approach to support planning the evolution of product lines.

Acknowledgements

We would like to thank Robert O. Briggs for his invaluable comments and deep insights which helped us to fine-tune the process. We would also like to thank the team of PROSOL (Martin Schwaiger, Christoph Greiner, Thomas Fischer, and Simon Jansky) for participating in the workshop, sharing with us their ideas, and giving us feedback. Florian Graf was instrumental in setting up the workshop infrastructure for that we are greatly obliged to him.

Appendix. Example of a ThinkLet

Evaluate: StrawPoll

Choose this thinkLet ...

... to measure consensus within a group.

... to reveal patterns of agreement or disagreement within a group.

... to assess or evaluate a set of concepts.

Overview

StrawPoll enables you to take the temperature of the group. It lets you quickly find out which preferences the group has and what the level of consensus is among the participants.

Capability

1. Prepare the appropriate ballots with issues and choices
2. Select a voting method
3. Collect the ballots and compute the results

Script

1. Say this:
 - (a) We are going to take a straw poll. We are not making a final decision right now. We just want to get a sense of the group so we can focus our subsequent efforts where they should be focused.
 - (b) I have sent you a ballot containing a set of X items.
 - (c) Please rate each item on a scale from Y to Z.
 - (d) A rating of Y means ...
 - (e) A rating of Z means ...
 - (f) When you are done voting, click the “submit ballot” button (incase of electronic tool) or put your ballots in this box (incase of traditional paper balloting)

References

- Atkinson, C. et al., 2002. Component-Based Product Line Engineering with UML. Addison-Wesley.
- Aversano, L., Tortorella, M., 2004. An assessment strategy for identifying legacy system evolution requirements in eBusiness context. *Journal of Software Maintenance and Evolution: Research Practice* 16, 255–276.
- Bayer, J. et al., 1999a. Transitioning legacy assets to a product line architecture. *ACM SIGSOFT Software Engineering Notes* 24 (6), 446–463.
- Bayer, J. et al., 1999b. PuLSE: a methodology to develop software product lines. In: *Proceedings of the 1999 Symposium on Software Reusability, collocated with the 1999 International Conference on Software Engineering (ICSE'99)*. ACM Press.
- Bayer, J. et al., 2004. Definition of Reference Architectures based on Existing Systems. *Fraunhofer IESE-Report No. 034.04/E*, 173.
- Boehm, B., 2002. Get ready for agile methods, with care. *IEEE Computer* 35 (1), 64–69.
- Boehm, B., Ross, R., 1989. Theory-W software project management principles and examples. *IEEE Transactions on Software Engineering* 15 (7), 902–916.

- Boehm, B., Jain, A., 2005. An initial theory of value-based software engineering. In: Biffl, S. et al. (Eds.), *Value-Based Software Engineering*. Springer, Berlin Heidelberg, pp. 15–37.
- Boehm, B., Port, D., Al-Said, M., 2000. Avoiding the software model-clash spiderweb. *IEEE Computer* 33 (11), 120–122.
- Bosch, J., 2000. *Design and Use of Software Architectures: Adopting and Evolving a Product-line Approach*. ACM Press, Addison-Wesley Professional.
- Briggs, R.O., Gruenbacher, P., 2002. EasyWinWin: managing complexity in requirements negotiation with GSS. In: *Proceedings of the 35th Annual Hawaii International Conference on System Sciences (HICSS02)*, vol. 1. IEEE CS, pp. 21–30.
- Briggs, R.O., de Vreede, G.-J., 2001. ThinkLets: Building Blocks for Concerted Collaboration; The Netherlands, Innovative Group Support.
- Briggs, R.O. et al., 2001. ThinkLets: achieving predictable, repeatable patterns of group interaction with group support systems (GSS). In: *Proceedings of the 34th Annual Hawaii International Conference on System Sciences (HICSS-34)*, vol. 1. IEEE CS, pp. 1057–1065.
- Briggs, R.O., de Vreede, G.J., Nunamaker Jr., J.F., 2003. Collaboration engineering with ThinkLets to pursue sustained success with group support systems. *Journal of Management Information Systems* 19 (4), 31–64.
- Briggs, R.O. et al., 2006. Defining key concepts for collaboration engineering. Presented at the 12th Americas Conference on Information Systems (AMCIS); Acapulco, Mexico.
- Carbon, R. et al., 2006. Integrating product line engineering and agile methods: flexible design up-front vs. incremental design. Presented at the 1st International Workshop on Agile Product Line Engineering (APLE), collocated with the 10th International Software Product Line Conference (SPLC); Baltimore, MD, USA.
- Clements, P., Northrop, L., 2001. *Software Product Lines: Practices and Patterns*. Addison-Wesley.
- deVreede, G.J., Kolschoten, G.L., Briggs, R.O., 2006. ThinkLets: a collaboration engineering pattern language. *International Journal of Computer Applications in Technology* 25 (2), 140–154.
- Dhungana, D., Rabiser, R., Grünbacher, P., 2006a. Coordinating multi-team variability modeling in product line engineering. In: *Proceedings of the 2nd International Workshop on Supporting Knowledge Collaboration in Software Development (KCS2006)*, collocated with the 21st IEEE International Conference on Automated Software Engineering (ASE'06) Tokyo, Japan. IEEE CS Press, pp. 60–62.
- Dhungana, D. et al., 2006b. Architectural knowledge in product line engineering: an industrial case study. In: *Proceedings of the 32nd EUROMICRO Conference on Software Engineering and Advanced Applications (EUROMICRO'06)*. IEEE CS Press, pp. 186–197.
- Ebert, C., Smouts, M., 2003. Tricks and traps of initiating a product line concept in existing products. In: *Proceedings of the 25th International Conference on Software Engineering (ICSE'03)*. IEEE CS Press, pp. 520–525.
- Grünbacher, P., 2000. *EasyWinWin OnLine: Moderators Guidebook, A Methodology for Negotiating Software Requirements*. USC-CSE, JKU Linz, GroupSystems.com.
- Grünbacher, P., Hofer, C., 2002. Complementing XP with requirements negotiation. In: *Proceedings of the 3rd International Conference on eXtreme Programming and Agile Processes in Software Engineering, Agile Alliance*, pp. 105–108.
- Kolschoten, G.L. et al., 2004. Recurring patterns of facilitation interventions in GSS sessions. In: *Proceedings of the 37th Annual Hawaii International Conference on System Sciences (HICSS'04)*. IEEE CS Press, pp. 19–28.
- Kolschoten, G.L. et al., 2005. ThinkLets as building blocks for collaboration processes: a further conceptualization. In: de Vreede, G.J., Guerrero, L.A., Raventós, G.M. (Eds.), *Groupware: Design, Implementation and Use*, 10th International Workshop, CRIWG 2004, San Carlos, Costa Rica, September 5–9, 2004, *Proceedings*. Springer, Berlin, Heidelberg, pp. 137–152.
- Kolschoten, G.L. et al., 2006. A conceptual foundation of the thinkLet concept for collaboration engineering. *International Journal of Human-Computer Studies* 64 (7), 611–621.
- Maiden, N. et al., 2007. Exploring scenario forms and ways of use to discover requirements on airports that minimize environmental impact. In: *Proceedings of RE'07: 15th IEEE International Requirements Engineering Conference*, New Delhi, India, IEEE CS, pp. 23–38.
- Navarrete, F., Botella, P., Franch, X., 2005. How agile COTS selection methods are (and can be)? In: *Proceedings of the 31st EUROMICRO Conference on Software Engineering and Advanced Applications (EUROMICRO-SEAA'05)*. IEEE CS Press, pp. 160–167.
- Noor, M.A., Rabiser, R., Grünbacher, P., 2006. A collaborative approach for reengineering-based product line scoping. Presented at the 1st International Workshop on Agile Product Line Engineering (APLE), collocated with the 10th International Software Product Line Conference (SPLC); Baltimore, MD, USA.
- Noor, M.A., Grünbacher, P., Briggs, R.O., 2007a. Defining a collaborative approach for product line scoping: a case study in collaboration engineering. In: *Proceedings of the 3rd IASTED Conference on Software Engineering (SE 2007)*, Product Lines and Components Track. ACTA Press, pp. 47–54.
- Noor, M.A., Grünbacher, P., Hoyer, C., 2007b. A collaborative method for reuse potential assessment in reengineering-based product line adoption. In: *2nd IFIP Central and East European Conference on Software Engineering Techniques (CEE-SET 07)*; Poznan, Poland.
- Northrop, L.M., 2004. *Software Product Line Adoption Roadmap*. CMU/SEI-2004-TR-022.
- Peterson, D.R., 2003. Economics of software product lines. In: van der Linden, F. (Ed.), *Software Product-Family Engineering*, 5th International Workshop, PFE 2003, Siena, Italy, November 4–6, 2003. Springer, Berlin Heidelberg, pp. 381–402 (Revised Papers).
- Pohl, K., Böckle, G., van der Linden, F., 2005. *Software Product Line Engineering: Foundations, Principles, and Techniques*. Springer.
- Rabiser, R., Dhungana, D., 2007. Integrated support for product configuration and requirements engineering in product derivation. In: *33rd EUROMICRO Conference on Software Engineering and Advanced Applications (EUROMICRO-SEAA'07)*; Lübeck, Germany. IEEE CS, pp. 219–228.
- Schmid, K., 2002a. A comprehensive product line scoping approach and its validation. In: *Proceedings of the 24th International Conference on Software Engineering*. ACM Press, pp. 593–603.
- Schmid, K., 2002b. The product line mapping approach to defining and structuring product portfolios. In: *Proceedings of the 10th Anniversary IEEE Joint International Conference on Requirements Engineering*. IEEE CS Press, pp. 219–226.
- Schmid, K., Verlage, M., 2002. The economic impact of product line adoption and evolution. *IEEE Software* 19 (4), 50–57.
- Stoermer, C., O'Brien, L., 2001. MAP-mining architectures for product line evaluations. In: *Proceedings of the Working IEEE/IFIP Conference on Software Architecture (WICSA'01)*. IEEE CS Press, pp. 35–44.
- Tian, K., Cooper, K., 2006. Agile and software product line methods: are they so different? Presented at the 1st International Workshop on Agile Product Line Engineering (APLE), collocated with the 10th International Software Product Line Conference (SPLC), Baltimore, MD, USA.
- Turner, R., Boehm, B., 2003. People factors in software management: lessons from comparing agile and plan-driven methods, crosstalk. *The Journal of Defense Software Engineering* 10 (12), 4–8.

Muhammad Asim Noor is a PhD student at the Institute for Systems Engineering and Automation at Johannes Kepler University Linz. Contact him at man@sea.uni-linz.ac.at.

Rick Rabiser is a PhD student and researcher in the Christian Doppler Laboratory for Automated Software Engineering at Johannes Kepler University Linz. Contact him at rabiser@ase.jku.at.

Paul Grünbacher is an associate professor at Johannes Kepler University Linz and senior researcher in the university's Christian Doppler Laboratory for Automated Software Engineering. Contact him at paul.gruenbacher@jku.at.